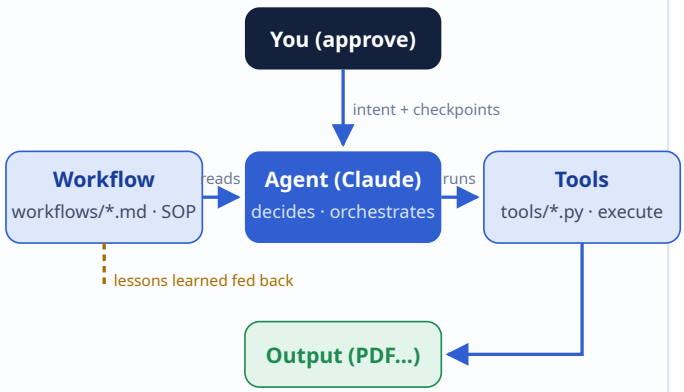
Claude Workflow Cheat Sheet

Build a repeatable, evidence-backed AI workflow in Claude Code from an empty folder: the loop, the phases, the prompts, the commands and the traps. Written from the competitor-analysis build (9 competitors, 192 sources, ~412 cited claims, validator-gated PDF).

WAT = Workflows · Agents · Tools | Plan → Build → Check → Approve | Every claim cites a source | Secrets only in .env

1 The core idea in one picture

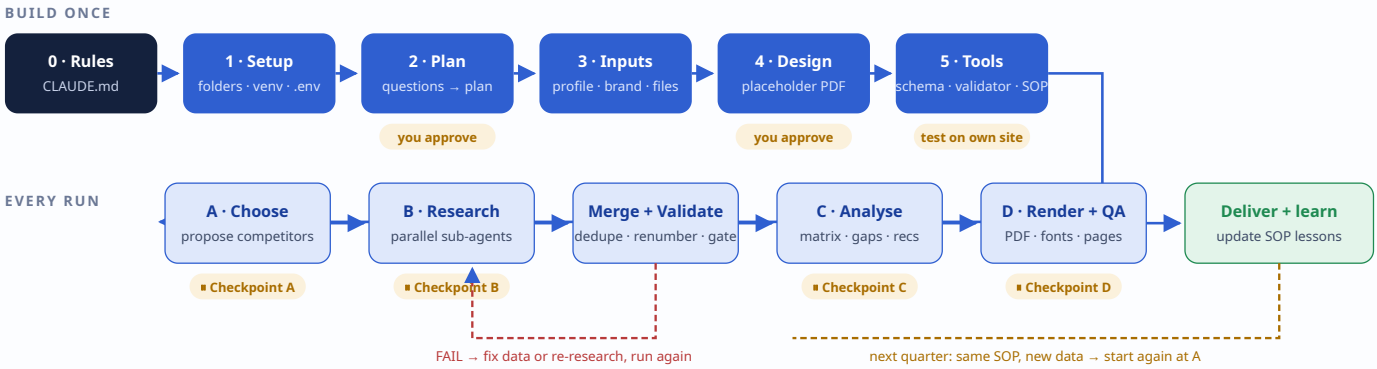


Why split the work

If each step is 90% reliable, five AI-only steps succeed 59% of the time. So the AI only **decides**; repeatable work runs as **code**; you **approve** at checkpoints.

- **Workflow**: plain-language SOP: goal, inputs, tools, outputs, edge cases, lessons learned.
- **Agent**: reads the SOP, runs tools in order, fixes failures, asks when unsure.
- **Tools**: deterministic Python scripts (API calls, merges, validation, rendering).
- **Gate**: a validator blocks the output if any rule is broken.

2 The build algorithm, from zero



Rule of thumb: **plan before code, design before research, validate before render, approve before the next phase.**

3 Step-by-step checklist

0 · Rules — 5 MIN

- Empty folder, `git init`
- Write `CLAUDE.md` (prompt 0)
- Decide where deliverables live

Out: rules loaded into every session

1 · Setup — 10 MIN

- `tools/ workflows/ .tmp/`
- `.env` + `.env.example`, `.gitignore`
- venv + `requirements.txt`

Out: working skeleton, no secrets in git

2 · Plan — APPROVE

- Switch to plan mode (`Shift` + `Tab`)
- Describe goal, inputs, output, structure
- Answer all questions in one batch

Out: phased plan with checkpoints

3 · Inputs CHECK

- `business-profile.md`
- `brand/` → `brand.json` (colours, fonts, voice)
- Internal files: verify every claim

Out: trusted inputs, conflicts resolved by you

4 · Design APPROVE

- Template with placeholder data
- Every section type rendered once
- Fonts embedded, no blank pages

Out: sample PDF you like

5 · Tools TEST

- JSON schema: claim = text + source_ids
- Validator, scrapers, speed, merge, charts, render
- SOP in `workflows/`

Out: each tool tested on your own site

A · Choose CHECKPOINT

- Claude proposes, with rationale + source
- Search for new entrants
- Others → "rest of market" table

Out: approved list

B · Research CHECKPOINT

- 1 sub-agent per company, same brief
- Local ids L1… → merge renumbers to S…
- Validator with `--check-links` until PASS

Out: evidence base you reviewed

C · Analyse → D · Render CHECKPOINT

- Matrix 1–5, positioning, gaps, recs (impact × effort)
- Executive summary **last**
- Render, view every page as image, sign off

Out: final PDF + lessons in SOP

4 Build prompts (send one at a time)

Replace the `[yellow parts]`. Review each result before sending the next.

0 Project rules save as CLAUDE.md

```
You work inside the WAT framework (Workflows, Agents,
Tools).
- Workflows: Markdown SOPs in workflows/ (objective,
inputs, tools, outputs, edge cases).
- Agent (you): read the workflow, run tools in order,
handle failures, ask when unsure.
- Tools: Python scripts in tools/ for deterministic
work. Secrets only in .env.
Rules:
1. Look for an existing tool before writing a new
one.
2. On failure: read the full error, fix the tool, re-
test, record the lesson in the
   workflow. Ask me before re-running anything that
costs money.
3. Don't overwrite a workflow without asking me.
4. Temp files in .tmp/ (disposable). Final outputs go
to [where].
```

1 Initialise first message

```
I just added a CLAUDE.md that explains how I want you
to work in this project.
Initialise the project and set everything up
(folders, .gitignore, .env.example,
requirements.txt, a Python virtual environment). Ask
me any questions first.
```

2 Describe + plan plan mode

```
I want to build a REPEATABLE [competitor analysis]
workflow.
The output of each run is a fully branded [PDF
report].
Inputs: business info (save to business-profile.md),
brand guidelines + logo.
The workflow should: [step 1 … step n].
Every claim must cite a source URL and access date.
Structure: [Cover → Summary → … → Sources]
Ask me ALL clarifying questions in one batch first.
Then give me a plan with
phases, files/folders, libraries and why, and a
checkpoint after each phase.
Don't write code until I approve.
```

3 Business profile

```
Here is my business profile. Save it to business-
profile.md exactly as written.
Then tell me what in it changes your plan, and list
anything that looks
inconsistent or needs a source.
[paste profile]
```

4 Brand

```
I put my brand guidelines, logo and fonts in brand/.
Extract colours, fonts,
type scale, logo rules, motif and tone of voice into
brand/brand.json.
Show me what you found and flag any rule that
conflicts with this report
before you use it.
```

5 Design first

```
Build the branded template now with placeholder data
only, no research yet.
Cover every section type once. Render a sample PDF,
check the brand fonts are
embedded and there are no blank pages, and show me
the result.
```

6 Verify internal files

```
I added business files in [folder]. Some info may be
outdated, so check
everything. Mark each external claim confirmed /
outdated / contradicted /
unverifiable, with the source link. For internal
facts you can't check online,
list the conflicts and ask me. Don't update the
profile until I answer.
```

7 Tools + SOP

```
Build the tools and the workflow SOP:
- a JSON schema (every claim carries source ids),
- a validator that blocks rendering on unsourced
claims, dead links, placeholders,
- [website snapshot, PageSpeed, …] tools,
- workflows/[name].md with steps, checkpoints,
evidence rules, edge cases.
Test every tool on my own website first and show me
the results.
```

8 Choose competitors Checkpoint A

```
Start the first run. Propose [5–8] direct and [2–3]
indirect competitors in
[market], each with a one-line rationale and a
source. Search for new entrants.
Put everyone else in a short "rest of the market"
table. Wait for my approval.
```

9 Parallel research Checkpoint B

```
Research each approved competitor in parallel, one
sub-agent per company, same
written brief: only cite pages actually opened, label
self-claims as claims,
label inference, say "not publicly disclosed" when
there is no evidence.
Run the website and speed tools on every site. Merge,
run the validator until it
passes, then show me the findings before writing any
analysis.
```

10 Analysis Checkpoint C

```
Write the analysis from the approved evidence only:
matrix 1–5, positioning map,
what works for competitors, their weak spots, market
gaps, ranked recommendations
with impact, effort and timeframe. Executive summary
last. Every claim cites its
evidence. Be honest about my weaknesses; readers are
[investors].
```

11 Polish + sign off Checkpoint D

```
Render the final PDF. Keep it to [30–50] pages, fonts
embedded, no blank pages,
every section filled; look at every page as an image
before handing it over.
Then add what we learned in this run to the
workflow's "lessons learned".
```

12 Re-run next quarter

```
Run workflows/[name].md for [month year]. Re-check
the facts in the latest
source check, compare with the previous run in
output/, and add a short
"what changed since last time" section. Stop at every
checkpoint as usual.
```

5 Everyday steering prompts

Where are we?

```
What's done, what's next, and
what do you need from me? Give me
the file paths.
```

Delegate a choice

```
You take the decision. State what
you chose and why in one line,
then continue.
```

Ask again

```
Ask me the questions again, one
batch, with your recommended
answer first.
```

Fix an error

```
This failed: [paste error]. Find
the root cause, fix the tool, re-
test,
and add the lesson to the
workflow. Don't re-run paid calls
without asking.
```

Fact-check

```
Re-open the source for every
claim in [section]. Remove or
rewrite any
claim the page doesn't support,
and list what you changed.
```

Too long

```
The PDF is [61] pages; target
[45]. Propose layout changes
first
(columns, caps, spacing) without
cutting evidence. Wait for my OK.
```

Visual QA

```
Render every page as an image,
look at each one, and list layout
problems
(blank pages, overflow, orphaned
headings, broken tables) before
fixing them.
```

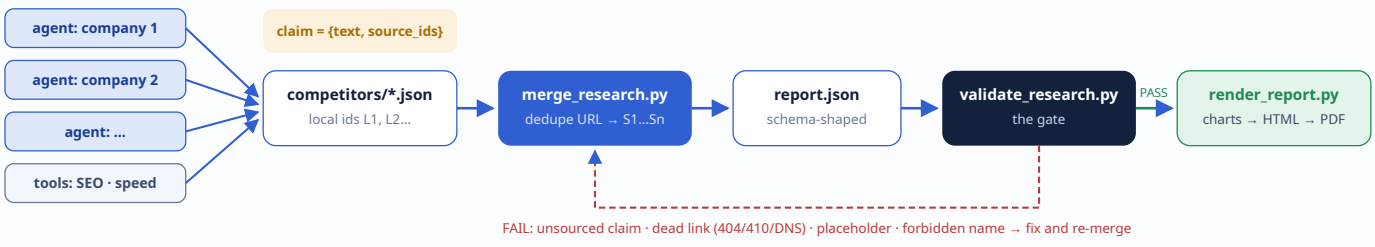
After /compact

```
Read the plan file and the
workflow, tell me where we
stopped, then continue.
```

Proof case

```
Build an HTML case study of this
workflow: architecture, algorithm
chart,
results, and the prompts to
rebuild it from zero. No
confidential data.
```

6 Evidence pipeline (how claims stay honest)



Evidence rules

- Only cite pages actually opened
- Self-claims labelled as claims
- Inference labelled
- "Not publicly disclosed"

Validator = errors

- Schema mismatch
- Missing source id
- No URL / future date
- 404, 410, DNS fail
- TODO, example.com, banned names

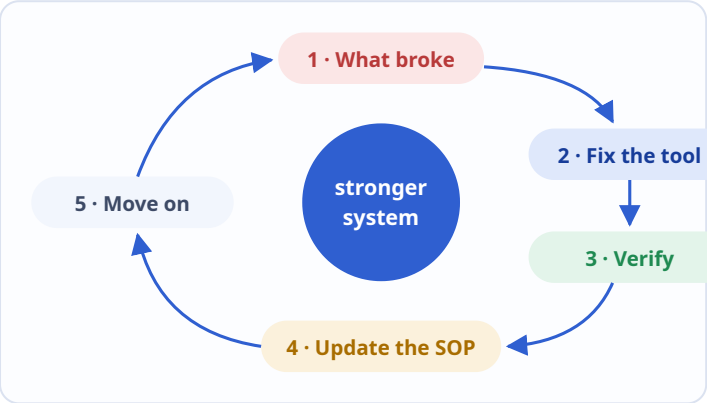
Validator = warnings

- 403 / 429 / SSL / timeout (bots)
- Source > 90 days old
- Unused sources
- Competitor count off target

Cross-references

Analysis in `base.json` can cite `"infobip:L1"`; merge resolves it to the global `S…` id.

7 When it breaks, plus Claude Code commands



Ask before re-running anything that costs credits. Write the lesson down, or the same bug returns next run.

COMMAND / KEY	USE IT FOR
`Shift` + `Tab`	Cycle permission modes, including **plan mode**
`Esc`	Interrupt Claude mid-task to redirect
`Esc` `Esc`	Jump back to an earlier message
`@path/file`	Point Claude at a specific file
`!command`	Run a shell command yourself, output goes into context
`/init`	Generate a starter CLAUDE.md from the repo
`/memory`	Edit CLAUDE.md / memory files
`/compact`	Summarise a long session to free context
`/clear`	Fresh context for an unrelated task
`/resume`	Reopen a previous session
`/model`	Switch model

8 Traps I hit (and the fix)

SYMPTOM	CAUSE	FIX
Brand font not in PDF	Jinja autoescape mangled the CSS	`{{ brand_css|safe }}` ; check with `pdffonts`
Blank first page	Running header element placed before content	Move it inside the first real section
Template crash on missing field	`StrictUndefined`	Use `.get()` for optional data
Citation number wraps alone	Line break before `<sup>`	Word joiner (U+2060) before the sup
Scraper hangs forever	Per-request timeout only	Wall-clock deadline + retry; line-parse robots.txt
Wait loop never ends	`pgrep -f` matches itself	Wait on the PID instead
Banned name false positive	Case-insensitive match on a common phrase	Case-sensitive + word boundary
PageSpeed quota errors	Keyless API calls	Free API key in `.env`, restricted to one API
Same source many ids	Agents number independently	Local ids + merge by normalised URL
Overstated claim	Wording stronger than evidence ("shows" vs "holds")	Re-open source, soften or drop
Dead link in sources	Cited a search snippet of a 404 page	Drop snippet-only evidence
Report too long	One-column sources, long tables	Two-column sources, cap long-tail rows

9 Project layout and run commands

```
project/
├─ CLAUDE.md            # rules, loaded every session
├─ business-profile.md  # who we are (reused every run)
├─ brand/               # guidelines, brand.json, fonts,
logo
├─ workflows/
│  └─ competitor_analysis.md  # the SOP + lessons
├─ tools/               # deterministic Python
├─ schemas/report.schema.json
├─ templates/           # report.html.j2 + report.css
├─ samples/             # placeholder data for design
├─ research/            # source checks
├─ output/<date_run>/   # data/, report.json, report.pdf
├─ .tmp/                # disposable
└─ .env                 # secrets only here (gitignored)
```

One run, end to end

```
RUN=output/2026-10-19_bd-a2p
python tools/seo_snapshot.py https://site1
https://site2 --out $RUN/data/seo.json
python tools/pagespeed.py https://site1 https://site2
--out $RUN/data/pagespeed.json
python tools/merge_research.py $RUN --order
id1,id2,id3
python tools/validate_research.py $RUN/report.json --
check-links --strict
python tools/render_report.py $RUN/report.json --out
$RUN/report.pdf
pdffonts $RUN/report.pdf | head            # brand
font embedded?
pdftoppm -r 40 -png $RUN/report.pdf .tmp/qa/p   #
page images for QA

# HTML → PDF (for this cheat sheet / case study)
google-chrome --headless=new --disable-gpu --no-pdf-
header-footer \
  --print-to-pdf=out.pdf file.html
```

10 Habits that made the difference

Plan before code

"Don't write code until I approve" keeps the build in order.

Questions in one batch

Answer once, up front, instead of being interrupted all build long.

Design before research

Approve a placeholder PDF so no research is wasted on layout churn.

Verify your own files

Old decks were wrong in places. Treat internal data as unverified.

Gate, don't trust

A validator that fails the build beats a promise to cite sources.

Parallel agents, one brief

Same written rules for every sub-agent gives comparable evidence.

Keep secrets out of chat

Put keys straight in `.env` ; restrict each key to one API.

Mark what's internal

Tell Claude what must never appear in output (prices, partner names).